

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №6

**Численное решение обыкновенных
дифференциальных уравнений**

Вариант №5

Преподаватель Наумова Надежда Александровна
Подготовил Лоскутов Прохор Александрович

Санкт - Петербург 2026

Оглавление

Оглавление	2
1 Цель работы	3
2 Порядок выполнения работы	3
3 Рабочие формулы методов	4
3.1 Постановка задачи Коши	4
3.2 Усовершенствованный метод Эйлера	4
3.3 Метод Рунге–Кутта 4-го порядка	4
3.4 Метод Милна	5
3.5 Оценка погрешности	5
4 Описание алгоритма решения задачи	6
4.1 Блок-схемы методов	8
4.2 Оценка точности по правилу Рунге	9
5 Вычислительный пример	10
6 Программная реализация задачи	11
6.1 Заголовочный файл <code>calc/ode.hpp</code>	12
6.2 Реализация <code>calc/ode.cpp</code>	13
7 Результаты выполнения программы	18
8 Выводы	20

1 Цель работы

Решить задачу Коши для обыкновенного дифференциального уравнения первого порядка $y' = f(x, y)$ с начальным условием $y(x_0) = y_0$ на отрезке $[x_0, x_n]$ численными методами. По варианту №5 для исследования используются **одношаговые** методы — усовершенствованный метод Эйлера и метод Рунге–Кутты 4-го порядка — и **многошаговый** метод предиктор-корректор — метод Милна. Для каждого метода строится таблица приближённых значений решения, оценивается погрешность (для одношаговых методов — по правилу Рунге, для метода Милна — по точному решению) и строятся графики точного и приближённого решений.

2 Порядок выполнения работы

1. Выбрать одно из предлагаемых программой уравнений вида $y' = f(x, y)$ (не менее трёх).
2. Задать с клавиатуры начальное условие $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h и точность ε .
3. Реализовать одношаговые методы (усовершенствованный Эйлера, Рунге–Кутты 4) и многошаговый метод (Милна) в виде отдельных функций численного ядра.
4. Составить таблицу приближённых значений интеграла уравнения для всех методов.
5. Для одношаговых методов оценить точность по правилу Рунге; для метода Милна — по точному решению $\varepsilon = \max_i |y_{i,\text{точн}} - y_i|$.
6. Построить графики точного и приближённого решений разными цветами.
7. Протестировать программу на различных наборах данных, в том числе некорректных.
8. Проанализировать результаты.

3 Рабочие формулы методов

3.1 Постановка задачи Коши

Требуется найти функцию $y = y(x)$, удовлетворяющую уравнению $y' = f(x, y)$ и принимающую в начальной точке заданное значение $y(x_0) = y_0$. Численное решение заменяет непрерывную функцию сеточной: на равномерной сетке узлов $x_i = x_0 + ih$, $i = 0, 1, \dots, n$, вычисляются приближённые значения $y_i \approx y(x_i)$. Значение y_0 известно из начального условия, остальные находятся по рекуррентным формулам выбранного метода.

3.2 Усовершенствованный метод Эйлера

Одношаговый метод второго порядка точности ($\delta_n = O(h^2)$), построенный по схеме «предиктор–корректор» с трапециевидной аппроксимацией. На предикторе вычисляется грубое приближение по обычному методу Эйлера, на корректоре оно уточняется средним наклоном:

$$\begin{aligned}\tilde{y}_{i+1} &= y_i + hf(x_i, y_i) \\ y_{i+1} &= y_i + \frac{h}{2}[f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})]\end{aligned}$$

3.3 Метод Рунге–Кутта 4-го порядка

Наиболее распространённый одношаговый метод четвёртого порядка точности ($\delta_n = O(h^4)$). На каждом шаге правая часть вычисляется в четырёх точках:

$$\begin{aligned}y_{i+1} &= y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 &= hf(x_i, y_i), \quad k_2 = hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right) \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), \quad k_4 = hf(x_i + h, y_i + k_3)\end{aligned}$$

3.4 Метод Милна

Многошаговый метод предиктор-корректор четвертого порядка ($\delta_n = O(h^4)$), основанный на первой интерполяционной формуле Ньютона с разностями до третьего порядка. Для запуска требуются значения в четырех первых узлах: y_0 задаётся начальным условием, а y_1, y_2, y_3 вычисляются методом Рунге–Кутты 4-го порядка. Далее при $i \geq 4$:

этап прогноза (предиктор):

$$y_i^{\text{прогн}} = y_{i-4} + \frac{4h}{3}(2f_{i-3} - f_{i-2} + 2f_{i-1})$$

этап коррекции (корректор):

$$y_i^{\text{корр}} = y_{i-2} + \frac{h}{3}(f_{i-2} + 4f_{i-1} + f_i^{\text{прогн}}), \quad f_i^{\text{прогн}} = f(x_i, y_i^{\text{прогн}})$$

Коррекция повторяется итеративно (предыдущая коррекция подставляется в правую часть) до выполнения условия $|y_i^{\text{корр}} - y_i^{\text{пред}}| < \varepsilon$, после чего происходит переход к следующему узлу.

3.5 Оценка погрешности

Правило Рунге (одношаговые методы). Задача решается дважды — с шагом h и с шагом $h/2$, после чего на конце отрезка вычисляется оценка погрешности

$$R = \frac{y^h - y^{h/2}}{2^p - 1},$$

где y^h и $y^{h/2}$ — значения на конце отрезка при шагах h и $h/2$, а p — порядок точности метода ($p = 2$ для усовершенствованного Эйлера, $p = 4$ для Рунге–Кутты). Если точность не достигнута ($R > \varepsilon$), шаг делится пополам, и вычисления повторяются — так осуществляется автоматический контроль точности.

Сравнение с точным решением (метод Милна). Для многошагового метода погрешность оценивается по точному решению задачи:

$$\varepsilon = \max_{0 \leq i \leq n} |y_{i,\text{точн}} - y_i|.$$

Для всех предлагаемых уравнений точное решение задачи Коши известно в замкнутом виде для произвольного начального условия (x_0, y_0) :

Таблица 1 — предлагаемые уравнения и их точные решения

$y' = f(x, y)$	Точное решение $y(x)$
$y' = y$	$y_0 e^{x-x_0}$
$y' = x + y$	$(y_0 + x_0 + 1)e^{x-x_0} - x - 1$
$y' = -2xy$	$y_0 e^{x_0^2 - x^2}$
$y' = x - y$	$(y_0 - x_0 + 1)e^{x_0 - x} + x - 1$

4 Описание алгоритма решения задачи

Общая процедура `solveCauchy` проверяет корректность входных данных, строит равномерную сетку $x_i = x_0 + ih$ и вызывает для неё все три метода, после чего формирует единую таблицу значений (точное решение и три приближённых) и сводку оценок точности. Блок-схема общей процедуры приведена на рис. 1.

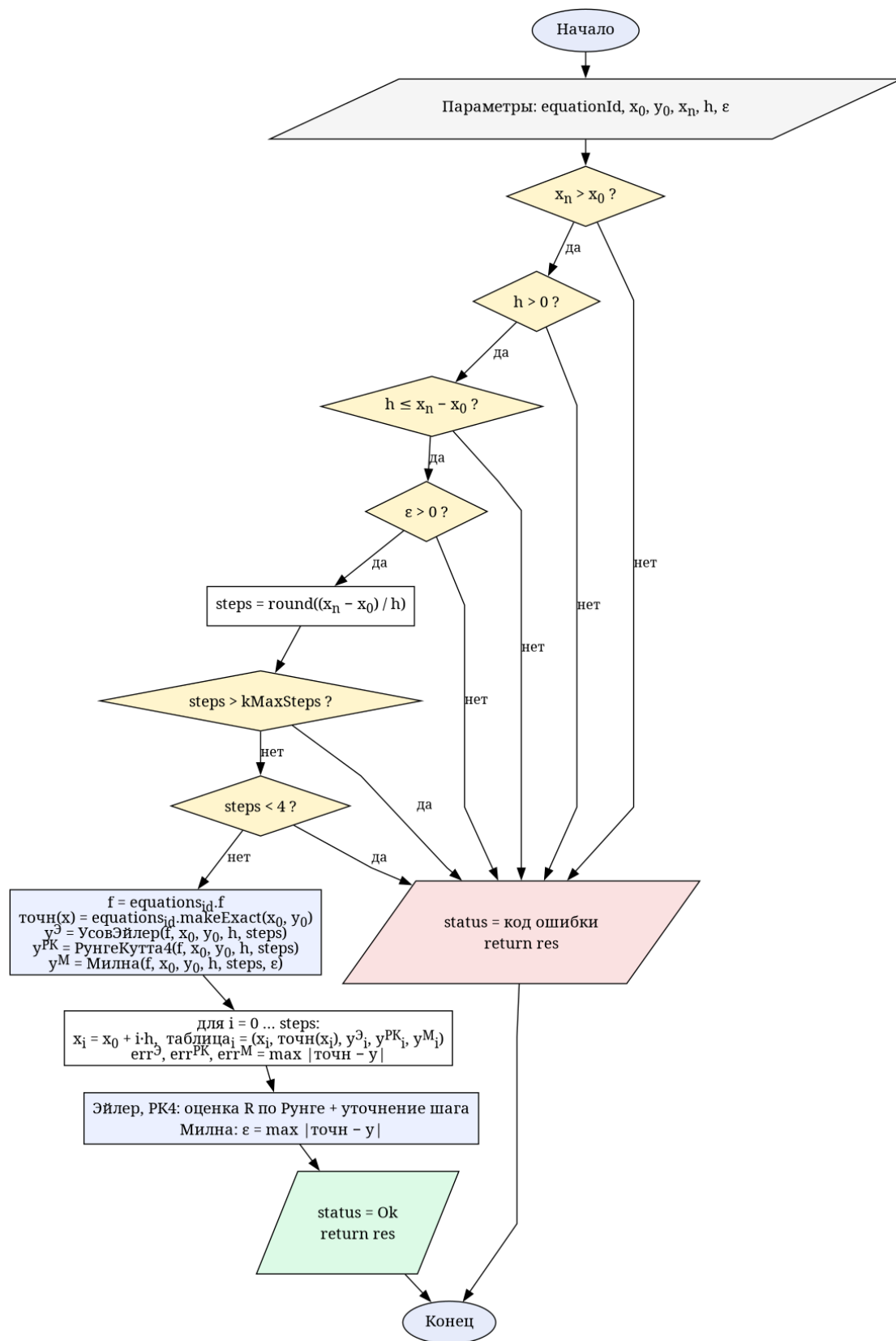


Рис. 1. Общая схема расчёта solveCauchy

4.1 Блок-схемы методов

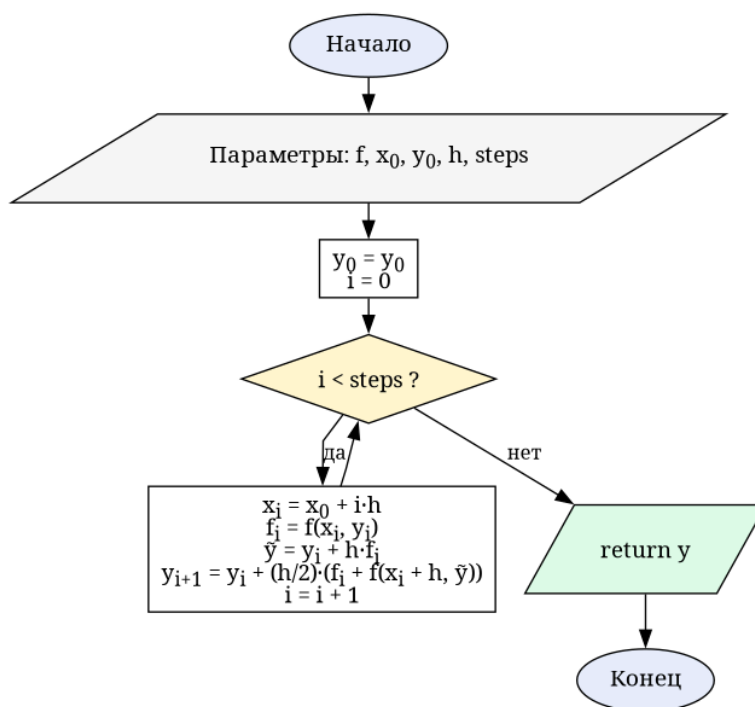


Рис. 2. Усовершенствованный метод Эйлера solveImprovedEuler

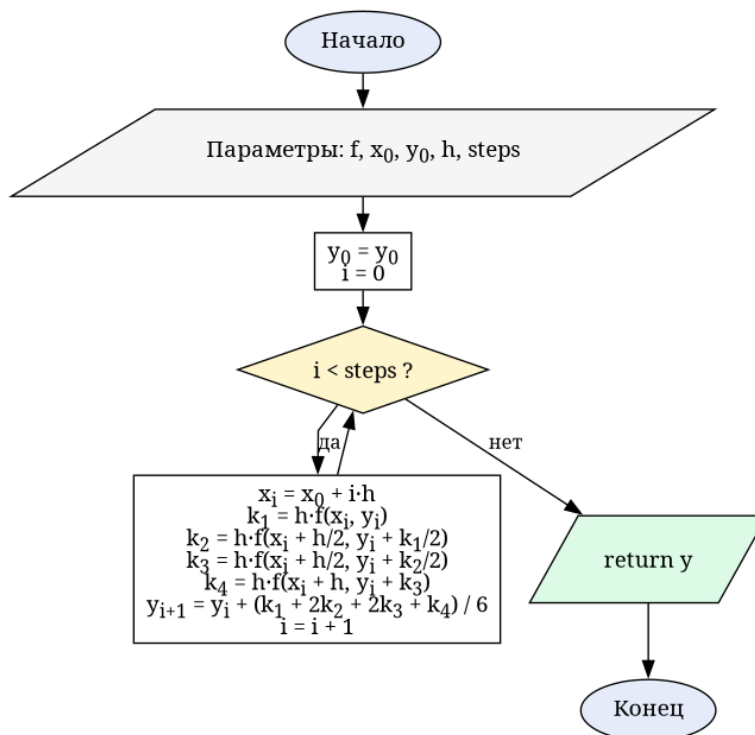


Рис. 3. Метод Рунге–Кутта 4-го порядка solveRungeKutta4

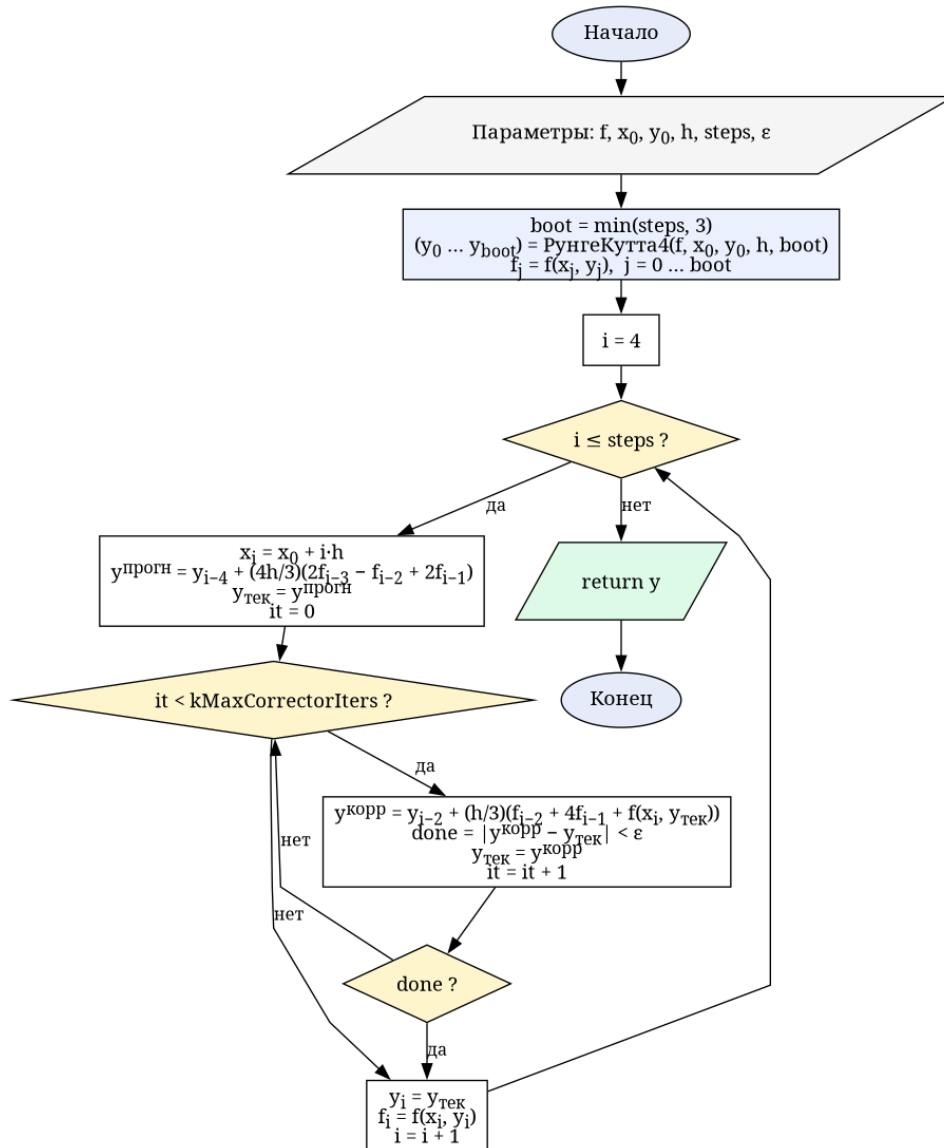


Рис. 4. Метод Милна (предиктор-корректор) `solveMilne`

4.2 Оценка точности по правилу Рунге

Контроль точности одношаговых методов с автоматическим дроблением шага реализован в функции `refineStepByRunge` (рис. 5): пока оценка R превышает ϵ , шаг уменьшается вдвое, число узлов удваивается, и оценка пересчитывается.

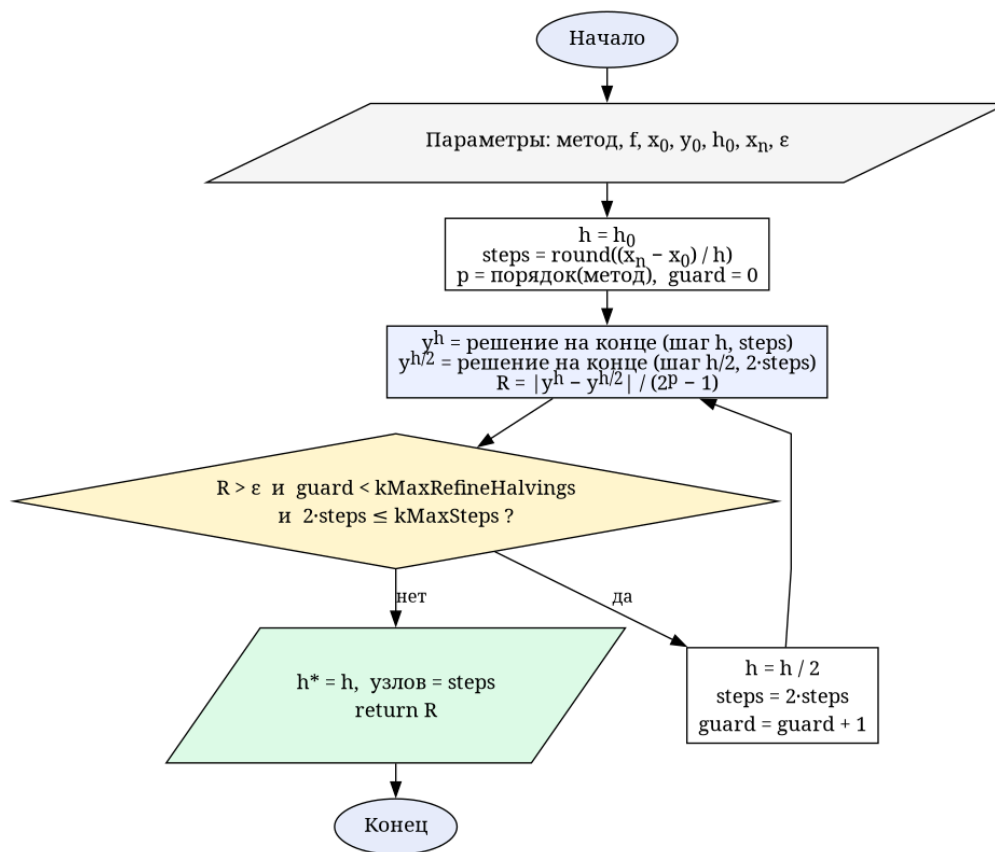


Рис. 5. Оценка погрешности и подбор шага по правилу Рунге
refineStepByRunge

5 Вычислительный пример

Рассмотрим задачу Коши $y' = y$, $y(0) = 1$ на отрезке $[0, 1]$ с шагом $h = 0.1$. Точное решение — $y(x) = e^x$. В таблице 2 приведены значения сеточной функции, полученные всеми тремя методами, и точное решение.

Таблица 2 — приближённые значения решения $y' = y$, $y(0) = 1$,
 $h = 0.1$

i	x_i	$y_{\text{точн}}$	$y_{\text{Эйлер}}$	$y_{\text{РК4}}$	$y_{\text{Милна}}$
0	0.0	1.000000	1.000000	1.000000	1.000000
1	0.1	1.105171	1.105000	1.105171	1.105171
2	0.2	1.221403	1.221025	1.221403	1.221403
3	0.3	1.349859	1.349233	1.349858	1.349858
4	0.4	1.491825	1.490902	1.491824	1.491824
5	0.5	1.648721	1.647447	1.648721	1.648721
6	0.6	1.822119	1.820429	1.822118	1.822119

7	0.7	2.013753	2.011574	2.013752	2.013752
8	0.8	2.225541	2.222789	2.225540	2.225541
9	0.9	2.459603	2.456182	2.459601	2.459603
10	1.0	2.718282	2.714081	2.718280	2.718282

Максимальная погрешность усовершенствованного метода Эйлера составляет $\approx 4.2 \cdot 10^{-3}$, метода Рунге–Кутта — $\approx 2.1 \cdot 10^{-6}$, метода Милна — $\approx 3.8 \cdot 10^{-7}$. Правило Рунге для усовершенствованного метода Эйлера даёт на конце отрезка оценку $R \approx 1.0 \cdot 10^{-3}$; так как при $h = 0.1$ точность $\varepsilon = 10^{-3}$ не достигнута, шаг автоматически дробится до $h^* = 0.05$ (20 узлов). Для метода Рунге–Кутта оценка $R \approx 1.3 \cdot 10^{-7}$ уже удовлетворяет ε , поэтому шаг остаётся равным 0.1. Это согласуется с порядками точности методов: $O(h^2)$ против $O(h^4)$.

6 Программная реализация задачи

Модуль «ОДУ» добавлен в общее приложение (Qt6, QML + C++). Пользователь выбирает одно из четырёх предлагаемых уравнений (таблица 1), задаёт начальное условие y_0 , отрезок $[x_0, x_n]$, шаг h и точность ε . Программа строит единую таблицу значений для всех методов, выводит для каждого метода оценку точности (правило Рунге с итоговым шагом для одношаговых методов и $\max |y_{\text{точн}} - y|$ для метода Милна) и отображает график: точное решение и три приближённых решения разными цветами.

Численное ядро (`calc/ode.hpp`, `calc/ode.cpp`) не зависит от Qt и оформлено отдельным пространством имён `ode`. Каждый метод реализован отдельной функцией; правая часть $f(x, y)$ и точное решение задаются для каждого уравнения лямбда-функциями. Перед расчётом проверяется корректность данных: $x_n > x_0$, $h > 0$, $h \leq x_n - x_0$, $\varepsilon > 0$, а также достаточность числа узлов для метода Милна (не менее четырёх шагов). При нарушении любого из условий возвращается понятное сообщение об ошибке, а таблица и график не строятся.

6.1 Заголовочный файл `calc/Ode.hpp`

```
#pragma once

#include <functional>
#include <vector>

namespace ode {

using Func = std::function<double(double x, double y)>;
using Exact = std::function<double(double x)>;

enum class Method { ImprovedEuler, RungeKutta4, Milne };

enum class Status {
    Ok,
    BadInterval,
    BadStep,
    StepTooLarge,
    BadTolerance,
    TooFewNodes,
    TooManyNodes
};

struct OdeEquation {
    int id = 0;
    Func f;
    std::function<Exact(double x0, double y0)> makeExact;
};

struct Node {
    double x = 0.0;
    double yExact = 0.0;
    double yImprovedEuler = 0.0;
    double yRungeKutta4 = 0.0;
    double yMilne = 0.0;
};

struct MethodSummary {
    Method method = Method::ImprovedEuler;
    int order = 2;
    bool usesRunge = true;
    double rungeError = 0.0;
    double exactError = 0.0;
    int steps = 0;
    double refinedStep = 0.0;
    int refinedSteps = 0;
    double refinedError = 0.0;
};

struct Result {
    Status status = Status::Ok;
    double x0 = 0.0;
    double xn = 0.0;
    double h = 0.0;
    double eps = 0.0;
    std::vector<Node> table;
    MethodSummary improvedEuler;
    MethodSummary rungeKutta4;
    MethodSummary milne;
};

}
```

```

};

constexpr int kMinSteps = 4;
constexpr int kMaxSteps = 100000;
constexpr int kMaxCorrectorIters = 20;
constexpr int kMaxRefineHalvings = 20;

const std::vector<OdeEquation> &equations();
const char *equationTitle(int id);
bool equationExists(int id);

std::vector<double> solveImprovedEuler(const Func &f, double x0, double y0,
                                     double h, int steps);
std::vector<double> solveRungeKutta4(const Func &f, double x0, double y0,
                                    double h, int steps);
std::vector<double> solveMilne(const Func &f, double x0, double y0, double h,
                              int steps, double eps);

int methodOrder(Method m);

double rungeEndpointError(Method m, const Func &f, double x0, double y0,
                          double h, int steps);
double refineStepByRunge(Method m, const Func &f, double x0, double y0,
                         double h0, double xn, double eps, double &outStep,
                         int &outSteps);

Result solveCauchy(int equationId, double x0, double y0, double xn, double h,
                  double eps);

const char *methodKey(Method m);
const char *methodTitle(Method m);

} // namespace ode

```

6.2 Реализация calc/ode.cpp

```

#include "Ode.hpp"

#include <algorithm>
#include <cmath>

namespace ode {

const std::vector<OdeEquation> &equations() {
    static const std::vector<OdeEquation> table = {
        {0, [] (double, double y) { return y; }},
        [] (double x0, double y0) {
            return Exact([x0, y0] (double x) { return y0 * std::exp(x - x0); });
        },
        {1, [] (double x, double y) { return x + y; }},
        [] (double x0, double y0) {
            const double c = y0 + x0 + 1.0;
            return Exact(
                [c, x0] (double x) { return c * std::exp(x - x0) - x - 1.0; });
        },
        {2, [] (double x, double y) { return -2.0 * x * y; }},
        [] (double x0, double y0) {
            return Exact(
                [x0, y0] (double x) { return y0 * std::exp(x0 * x0 - x * x); });
        }
    };
}

```

```

    }},
    {3, [](double x, double y) { return x - y; },
    [](double x0, double y0) {
        const double c = y0 - x0 + 1.0;
        return Exact(
            [c, x0](double x) { return c * std::exp(x0 - x) + x - 1.0; });
    }},
};
return table;
}

bool equationExists(int id) {
    return id ≥ 0 && id < static_cast<int>(equations().size());
}

const char *equationTitle(int id) {
    switch (id) {
    case 0:
        return "y' = y";
    case 1:
        return "y' = x + y";
    case 2:
        return "y' = -2xy";
    case 3:
        return "y' = x - y";
    }
    return "";
}

std::vector<double> solveImprovedEuler(const Func &f, double x0, double y0,
                                     double h, int steps) {
    std::vector<double> y(static_cast<std::size_t>(steps) + 1);
    y[0] = y0;
    for (int i = 0; i < steps; ++i) {
        const double xi = x0 + i * h;
        const double fi = f(xi, y[i]);
        const double predictor = y[i] + h * fi;
        y[i + 1] = y[i] + 0.5 * h * (fi + f(xi + h, predictor));
    }
    return y;
}

std::vector<double> solveRungeKutta4(const Func &f, double x0, double y0,
                                     double h, int steps) {
    std::vector<double> y(static_cast<std::size_t>(steps) + 1);
    y[0] = y0;
    for (int i = 0; i < steps; ++i) {
        const double xi = x0 + i * h;
        const double k1 = h * f(xi, y[i]);
        const double k2 = h * f(xi + 0.5 * h, y[i] + 0.5 * k1);
        const double k3 = h * f(xi + 0.5 * h, y[i] + 0.5 * k2);
        const double k4 = h * f(xi + h, y[i] + k3);
        y[i + 1] = y[i] + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0;
    }
    return y;
}

std::vector<double> solveMilne(const Func &f, double x0, double y0, double h,
                              int steps, double eps) {
    std::vector<double> y(static_cast<std::size_t>(steps) + 1);

```

```

std::vector<double> fv(static_cast<std::size_t>(steps) + 1);

const int boot = std::min(steps, 3);
const std::vector<double> start = solveRungeKutta4(f, x0, y0, h, boot);
for (int i = 0; i ≤ boot; ++i) {
    y[i] = start[i];
    fv[i] = f(x0 + i * h, y[i]);
}

for (int i = 4; i ≤ steps; ++i) {
    const double xi = x0 + i * h;
    const double yPred =
        y[i - 4] +
        (4.0 * h / 3.0) * (2.0 * fv[i - 3] - fv[i - 2] + 2.0 * fv[i - 1]);
    double yCur = yPred;
    for (int it = 0; it < kMaxCorrectorIters; ++it) {
        const double yCorr =
            y[i - 2] + (h / 3.0) * (fv[i - 2] + 4.0 * fv[i - 1] + f(xi, yCur));
        const bool done = std::abs(yCorr - yCur) < eps;
        yCur = yCorr;
        if (done) {
            break;
        }
    }
    y[i] = yCur;
    fv[i] = f(xi, yCur);
}
return y;
}

int methodOrder(Method m) {
    switch (m) {
        case Method::ImprovedEuler:
            return 2;
        case Method::RungeKutta4:
            return 4;
        case Method::Milne:
            return 4;
    }
    return 1;
}

namespace {

double solveEndpoint(Method m, const Func &f, double x0, double y0, double h,
                    int steps) {
    std::vector<double> y;
    switch (m) {
        case Method::ImprovedEuler:
            y = solveImprovedEuler(f, x0, y0, h, steps);
            break;
        case Method::RungeKutta4:
            y = solveRungeKutta4(f, x0, y0, h, steps);
            break;
        case Method::Milne:
            y = solveMilne(f, x0, y0, h, steps, 1e-12);
            break;
    }
    return y.empty() ? y0 : y.back();
}

```

```

} // namespace

double rungeEndpointError(Method m, const Func &f, double x0, double y0,
                          double h, int steps) {
    const double yH = solveEndpoint(m, f, x0, y0, h, steps);
    const double yH2 = solveEndpoint(m, f, x0, y0, h / 2.0, steps * 2);
    const double denom = std::pow(2.0, methodOrder(m)) - 1.0;
    return std::abs(yH - yH2) / denom;
}

double refineStepByRunge(Method m, const Func &f, double x0, double y0,
                        double h0, double xn, double eps, double &outStep,
                        int &outSteps) {
    double h = h0;
    int steps = static_cast<int>(std::lround((xn - x0) / h));
    if (steps < 1) {
        steps = 1;
    }
    double r = rungeEndpointError(m, f, x0, y0, h, steps);
    int guard = 0;
    while (r > eps && guard < kMaxRefineHalvings && steps * 2 ≤ kMaxSteps) {
        h *= 0.5;
        steps *= 2;
        r = rungeEndpointError(m, f, x0, y0, h, steps);
        ++guard;
    }
    outStep = h;
    outSteps = steps;
    return r;
}

namespace {

MethodSummary makeSummary(Method m, const Func &f, double x0, double y0,
                        double xn, double h, double eps, int steps,
                        double exactError) {
    MethodSummary s;
    s.method = m;
    s.order = methodOrder(m);
    s.steps = steps;
    s.exactError = exactError;
    s.usesRunge = m ≠ Method::Milne;
    if (s.usesRunge) {
        s.rungeError = rungeEndpointError(m, f, x0, y0, h, steps);
        s.refinedError = refineStepByRunge(m, f, x0, y0, h, xn, eps, s.refinedStep,
                                           s.refinedSteps);
    }
    return s;
}

} // namespace

Result solveCauchy(int equationId, double x0, double y0, double xn, double h,
                  double eps) {
    Result res;
    res.x0 = x0;
    res.xn = xn;
    res.h = h;
    res.eps = eps;

```

```

if (!equationExists(equationId)) {
    res.status = Status::BadInterval;
    return res;
}
if (!(xn > x0)) {
    res.status = Status::BadInterval;
    return res;
}
if (!(h > 0.0)) {
    res.status = Status::BadStep;
    return res;
}
const double span = xn - x0;
if (h > span + 1e-12) {
    res.status = Status::StepTooLarge;
    return res;
}
if (!(eps > 0.0)) {
    res.status = Status::BadTolerance;
    return res;
}

int steps = static_cast<int>(std::lround(span / h));
if (steps < 1) {
    steps = 1;
}
if (steps > kMaxSteps) {
    res.status = Status::TooManyNodes;
    return res;
}
if (steps < kMinSteps) {
    res.status = Status::TooFewNodes;
    return res;
}

const OdeEquation &eq = equations()[static_cast<std::size_t>(equationId)];
const Func &f = eq.f;
const Exact exact = eq.makeExact(x0, y0);

const std::vector<double> yIE = solveImprovedEuler(f, x0, y0, h, steps);
const std::vector<double> yRK = solveRungeKutta4(f, x0, y0, h, steps);
const std::vector<double> yML = solveMilne(f, x0, y0, h, steps, eps);

res.table.resize(static_cast<std::size_t>(steps) + 1);
double errIE = 0.0;
double errRK = 0.0;
double errML = 0.0;
for (int i = 0; i ≤ steps; ++i) {
    const double xi = x0 + i * h;
    const double ye = exact(xi);
    Node &nd = res.table[static_cast<std::size_t>(i)];
    nd.x = xi;
    nd.yExact = ye;
    nd.yImprovedEuler = yIE[i];
    nd.yRungeKutta4 = yRK[i];
    nd.yMilne = yML[i];
    errIE = std::max(errIE, std::abs(ye - yIE[i]));
    errRK = std::max(errRK, std::abs(ye - yRK[i]));
    errML = std::max(errML, std::abs(ye - yML[i]));
}

```

```

}

res.improvedEuler =
    makeSummary(Method::ImprovedEuler, f, x0, y0, xn, h, eps, steps, errIE);
res.rungeKutta4 =
    makeSummary(Method::RungeKutta4, f, x0, y0, xn, h, eps, steps, errRK);
res.milne = makeSummary(Method::Milne, f, x0, y0, xn, h, eps, steps, errML);

res.status = Status::Ok;
return res;
}

const char *methodKey(Method m) {
    switch (m) {
        case Method::ImprovedEuler:
            return "improved_euler";
        case Method::RungeKutta4:
            return "rk4";
        case Method::Milne:
            return "milne";
    }
    return "";
}

const char *methodTitle(Method m) {
    switch (m) {
        case Method::ImprovedEuler:
            return "Усоверш. метод Эйлера";
        case Method::RungeKutta4:
            return "Метод Рунге-Кутта 4";
        case Method::Milne:
            return "Метод Милна";
    }
    return "";
}

} // namespace ode

```

7 Результаты выполнения программы

Программа протестирована на нескольких наборах данных, включая некорректные.

Модуль «ОДУ» — задача Коши

Набор данных №1

$$y' = y,$$

$[0, 1]$, шаг $h = 0.1$, точность $\varepsilon = 0.001$

Здесь будет реальный скриншот приложения — замените файл resources/lab6_app1.png

Рис. 6. Решение задачи $y' = y$, $[0, 1]$, $h = 0.1$, $\varepsilon = 0.001$

Модуль «ОДУ» — задача Коши

Набор данных №2

$$y' = -2xy,$$

$[0, 1]$, шаг $h = 0.1$, точность $\varepsilon = 0.001$

Здесь будет реальный скриншот приложения — замените файл resources/lab6_app2.png

Рис. 7. Решение задачи $y' = -2xy$, $[0, 1]$, $h = 0.1$ (гауссов спад)

Модуль «ОДУ» — задача Коши

Набор данных №3

Проверка на некорректных данных:
шаг h больше длины отрезка $[x_0, x_n]$

Здесь будет реальный скриншот приложения — замените файл resources/lab6_app3.png

Рис. 8. Реакция на некорректные данные: шаг h больше длины отрезка

На графиках точное решение и приближённые решения трёх методов отображаются разными цветами; кривые Рунге–Кутта и Милна практически сливаются с точным решением, а усовершенствованный метод Эйлера заметно отклоняется при больших шагах, что наглядно иллюстрирует разницу порядков точности.

Полный исходный код проекта (Qt6/QML/C++ десктоп-приложение) доступен в репозитории:

`git.oriptal.dev/cadmin/calc-math-lab2`

8 Выводы

В ходе лабораторной работы изучены и программно реализованы численные методы решения задачи Коши для обыкновенных дифференциальных уравнений: усовершенствованный метод Эйлера (второго порядка), метод Рунге–Кутта 4-го порядка и многошаговый метод Милна типа предиктор-корректор (четвёртого порядка). Для одношаговых методов реализован контроль точности по правилу Рунге с автоматическим дроблением шага, для метода Милна — сравнение с точным решением. Вычислительный эксперимент подтвердил теоретические порядки точности: метод Рунге–Кутта

и метод Милна дают погрешность на 3–4 порядка меньше, чем усовершенствованный метод Эйлера при том же шаге. Программа строит единую таблицу значений и графики точного и приближённых решений, корректно обрабатывает некорректные входные данные и согласована по стилю с остальными модулями приложения.